Perfect Matching Cut on Bipartite Graphs of Radius Three

Research manuscript — awaiting expert review
15 September 2026

Abstract

We give a deterministic polynomial-time many-one reduction from positive Not-All-Equal 3-SAT to Perfect Matching Cut on connected simple bipartite graphs of radius at most three. The construction combines a three-choice selector with binary variable interfaces, all sharing one center edge. Each interface uses its required crossing edges internally, so clause attachments enforce color equality without consuming additional crossing edges. For a nonempty formula with n variables and m clauses, the output has $4 + 8n + 160m$ vertices and $3 + 14n + 304m$ edges. We prove both directions and give polynomial-time recovery of a satisfying source assignment from every valid target coloring. The result addresses the radius-three gap between known radius-two tractability and radius-four hardness. Computational checks support the implementation; the arbitrary-input claim rests on the general proof.

1. Introduction

A perfect matching cut partitions the vertices of a graph into two nonempty sets so that every vertex has exactly one neighbor in the other set. This condition couples a local matching requirement with a global partition. Its complexity can change when the graph has small radius, because a center then constrains every vertex through a short path.

Lucke, Paulusma and Ries proved polynomial-time solvability for graphs of radius at most two [2, Theorem 21]. Lucke established hardness on bipartite graphs of radius four and explicitly asked for the radius-three case [1, Theorem 19 and Open Problem 2]. The radius-four construction uses cube clauses and connectors. The construction here realizes signed exact-one clauses with selectors sharing a center edge. Binary interfaces supply both a variable and its complement at distance two from the center.

Theorem 1. *There is a deterministic polynomial-time Karp reduction from positive NAE-3-SAT to Perfect Matching Cut on connected simple bipartite graphs of radius at most three. A deterministic polynomial-time algorithm recovers a satisfying source assignment from every perfect-cut coloring of the constructed graph. Consequently, Perfect Matching Cut is NP-complete on this graph class.*

The main construction requirement is that an interface retain its meaning when attached to the rest of the graph. We prove each relation with an upper bound of one internal crossing edge at its attachment vertices. The proof then forces those vertices to use exactly one internally, leaving every added incident edge monochromatic. This permits repeated occurrences of a variable without creating new copies of its interface.

2. Definitions and source encoding

For a graph $G = (V, E)$, write $N_G(v)$ for the neighbors of v. A perfect-cut coloring is a map $c : V \to \{0, 1\}$ such that

$$|\{w \in N_G(v) : c(w) \neq c(v)\}| = 1 \quad \text{for every } v \in V. \tag{1}$$

The color classes are automatically nonempty. The radius is the minimum, over vertices, of the greatest distance to any vertex. All constructed graphs are finite, undirected and simple.

A positive NAE-3-SAT instance is a formula Φ over n variables, with each clause containing three distinct unnegated variables. Each clause must contain both truth values. Repeated clauses and unused variables are allowed. This source is NP-complete [1, Theorem 18]. Let $E(l_1, l_2, l_3)$ mean that exactly one of the three signed literals is true.

Replace each clause (a, b, c) by the two disjunctions

$$(a \vee b \vee c) \wedge (\neg a \vee \neg b \vee \neg c). \tag{2}$$

For each disjunction $(l_1 \vee l_2 \vee l_3)$, introduce fresh bits $s_1, s_2, s_3, t_1, t_2, t_3$ and impose

$$E(s_1, s_2, s_3), \quad E(s_i, \neg l_i, t_i) \quad (i = 1, 2, 3). \tag{3}$$

Interpreting literal values as bits, the second condition gives

$$s_i + (1 - l_i) + t_i = 1, \quad t_i = l_i - s_i, \quad s_i \le l_i. \tag{4}$$

The unique true selector therefore chooses a true literal. Conversely, choose the first true literal in the displayed order and set the slack bits by this equation. The fresh variables make these extensions independent across clauses. The resulting signed exact-one formula Ψ has

$$N = n + 12m \quad \text{variables and} \quad C = 8m \quad \text{clauses}. \tag{5}$$

Every satisfying assignment of Ψ restricts to a satisfying assignment of Φ, and every satisfying assignment of Φ has the specified deterministic extension. No originality is claimed for this supporting Boolean encoding.

3. Graph interfaces

Start with four vertices u, h, b, t and edges uh, ub, bt. Every variable and clause block below has fresh private vertices. Only these named shared vertices and the stated attachment ports can have neighbors outside their blocks. Normalize a target coloring by complementing it if necessary so that $c(u) = 0$.

3.1. The selector

For a clause of Ψ, add vertices $F, A, d_1, d_2, d_3, T_1, T_2, T_3$ and edges

$$\{hF, uA\} \cup \{FT_i, Ad_i : i \in \{1, 2, 3\}\} \cup \{d_i T_j : i, j \in \{1, 2, 3\}, i \neq j\}. \tag{6}$$

Together with the shared edge uh, these form the ten-vertex graph in Figure 1. The vertices T_i are the clause attachment ports.

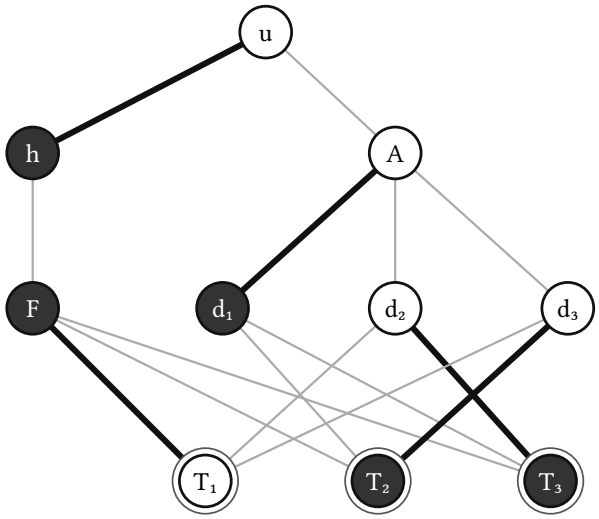


Figure 1: The exact selector graph, including the shared center edge uh. White and dark vertices have colors zero and one. Thick edges cross the cut; thin edges are monochromatic. Double circles mark the attachment ports T_i. The displayed cut selects $d_1 = 1$, and each T_i has already used its unique crossing edge. All internal edges are shown.

Lemma 2 (robust selector). *Suppose $c(u) = 0$, vertices F, A, d_1, d_2, d_3 have exactly one internal opposite neighbor, and vertices u, h, T_1, T_2, T_3 have at most one. Then $c(h) = c(F) = 1, c(A) = 0$, exactly one d_i has color one, and $c(T_i) = 1 - c(d_i)$. Each T_i has exactly one internal opposite neighbor. Every one-hot choice of the d_i extends to such a coloring.*

Proof. If $c(A) = 1$, the crossing edge Au forces all d_i to have color one. Each d_i then needs exactly one zero-colored neighbor among the other two T vertices. Summing these three requirements gives twice the number of zero-colored T vertices equal to three, a contradiction. Thus $c(A) = 0$. Since A needs exactly one opposite neighbor, exactly one d_i, say d_k, has color one.

The edge $d_k A$ already crosses, forcing $c(T_j) = 1$ for every $j \neq k$. Each remaining d_j has color zero and sees one such one-colored T vertex, so it forces $c(T_k) = 0$. Vertex F cannot have color zero because it would have two opposite neighbors. It therefore has color one, uses FT_k as its crossing edge, and forces $c(h) = 1$. Now T_k crosses only to F; each other T_j crosses only to the remaining zero-colored d vertex. These colors also verify the converse for each choice of k. □

The lemma applies to the restriction of any full perfect-cut coloring: attachment vertices can only lose crossing neighbors when outside edges are removed. It forces the center edge uh to cross without assuming that edge in advance.

3.2. The binary variable

For each variable of Ψ, add vertices f, p, r, z, a, v, w, q with edges

$$\{hf, fp, fr, zb, zp, zr, ua, av, aw, qv, qw, pv, rw, za\}. \tag{7}$$

The vertex p represents the bit, and r represents its complement. Ports v, w carry these same values. Figure 2 shows the block with its shared support.

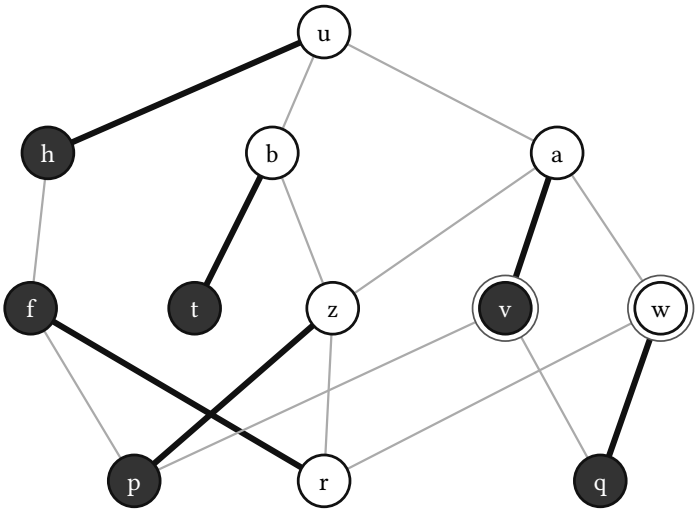


Figure 2: The binary variable graph with shared vertices u, h, b, t. All edges of the block and its shared support are shown. The displayed coloring has $p = v = 1$ and $r = w = 0$. Double circles mark occurrence ports; thick edges form the perfect cut. Every additional edge at v or w must be monochromatic.

Lemma 3 (binary interface). *In a perfect-cut coloring of the full construction with $c(u) = 0, c(h) = 1$, each variable block has $c(p) = c(v)$ and $c(r) = c(w) = 1 - c(p)$. Both occurrence ports already have one internal opposite neighbor. Conversely, either value of $c(p)$ extends to the block with these properties.*

Proof. The crossing edge uh forces $c(a) = c(b) = 0$ and $c(f) = 1$. The leaf t forces bt to cross, so $c(t) = 1$ and every neighbor z of b has color zero. Suppose $c(q) = 0$. A one-colored occurrence would have both a and q as opposite neighbors, so both occurrences must be zero; this leaves q without an opposite neighbor. Therefore $c(q) = 1$.

Each occurrence sees the fixed opposite colors at a and q, so it already uses one crossing edge internally. Thus pv and rw are monochromatic. Vertex a, whose other neighbors u, z are zero, forces exactly one of v, w to have color one. This proves the required relations.

Conversely, choose either value of $p = v$ and give $r = w$ its complement. Set $f = q = h = t = 1$ and $z = a = b = u = 0$. Each private vertex then has exactly one opposite neighbor: in particular p, r see the opposite colors of f, z, and f, z, a, q each see exactly one opposite member of the complementary pair. Shared b crosses only to t, and u, h cross only to each other. Additional monochromatic edges at v, w preserve every requirement. □

4. Clause composition and recovery

For the ith literal of a clause, connect its selector port T_i to w_x if the literal is x, and to v_x if the literal is $\neg x$. The same occurrence pair can serve arbitrarily many clauses. The attachment rule is

$$T_i w_x \quad \text{for } l_i = x, \quad T_i v_x \quad \text{for } l_i = \neg x. \tag{8}$$

Lemmas 2 and 3 force these edges to be monochromatic. In both cases, the color at the occurrence endpoint is the complement of the literal value. Therefore $c(d_i)$ equals that literal value, and the selector imposes exactly one true literal in the clause.

Proof of Theorem 1. First suppose Φ is satisfiable. Extend its assignment to Ψ by Equation 3. Color each binary block according to its encoded bit, and color each selector according to the unique true literal of its clause. The attachment edges are monochromatic. Lemmas 2 and 3 verify all private vertices, while the shared vertices use only uh and bt as their crossing edges. This gives a perfect cut of the constructed graph.

Conversely, let a perfect-cut coloring of the constructed graph be given. For a nonempty source formula there is at least one selector. After normalizing u to zero, Lemma 2 forces h to one, and Lemma 3 gives a consistent bit for every variable. By Equation 8, every selector certifies a signed exact-one clause. The decoded assignment satisfies Ψ and hence its restriction satisfies Φ. This applies to every target witness, including color complements.

The executable labels u by zero and the original variable port p_i by $5 + 8i$, with original variables indexed from zero. Recovery is the deterministic formula

$$g(\Phi, y)_i = y_{5+8i} \oplus y_0 \quad (0 \le i < n). \tag{9}$$

It checks the input witness on the reconstructed target and performs no source search or oracle call. The target-domain and polynomial bounds are proved below. Finally, a coloring and the target graph properties can be verified in polynomial time, establishing NP membership. □

4.1. Empty and invalid encodings

An empty formula maps to K_2; recovery returns n zero bits. The source encoding includes the explicit list of variables, so unused variables cannot be hidden in a short binary parameter. Invalid source strings map to C_6. This graph is connected, bipartite, and has radius three, but has no perfect matching cut: such a cut would contain three edges of the cycle, whereas every cut crosses a cycle an even number of times. Invalid recovery premises fail explicitly. These rules complete both algorithms on the specified encodings.

5. Radius and resource bounds

The following layers give paths from u and certify bipartiteness. Names range over all blocks of the indicated type.

Layer	Vertices
0	u
1	h, b, all a, A
2	t, all f, z, v, w, F, d_i
3	all p, r, q, T_i

Every edge in Equation 6, Equation 7 and Equation 8 joins adjacent layers, and every vertex above layer zero has a neighbor in the preceding layer. Thus the graph is connected and bipartite, and every vertex is within distance three of u. Private labels are distinct across blocks. The listed internal edges are distinct, and each attachment has its own clause port, so no loop or duplicate edge is introduced, even with repeated clauses.

For a nonempty source, the exact counts are

$$\begin{aligned} |V(G)| &= 4 + 8N + 8C = 4 + 8n + 160m, \\ |E(G)| &= 3 + 14N + 17C = 3 + 14n + 304m. \end{aligned} \tag{10}$$

The forward algorithm performs $O(n + m)$ graph operations and emits $O((n + m)\log(n + m + 1))$ bits. Label arithmetic and parsing are polynomial in the source encoding length L; a conservative bound is $O(L^2)$ bit operations. Recovery reconstructs the graph, checks each edge once and applies Equation 9. It uses at most $O((L + |y|)^2)$ bit operations and emits n bits. Both maps are deterministic and use no solver or persistent process state.

6. Scope of the result

The reduction supplies the radius-three hardness needed alongside the known radius-two algorithm for Perfect Matching Cut on bipartite graphs. Its shared center and saturated clause ports are the mechanism that keeps every vertex within three layers. This result does not imply the same classification for ordinary Matching Cut or Disconnected Perfect Matching, whose witness predicates differ from Equation 1.

The theorem and its claimed originality remain subject to expert assessment. The primary-literature check dated 15 September 2026 found no prior resolution within its recorded coverage. Unpublished, unindexed, inaccessible and overlooked work remain outside that coverage. Finite computational evidence and agent review do not constitute publication acceptance.

References

[1] Felicia Lucke. *Matching Cut and Variants on Bipartite Graphs of Bounded Radius and Diameter.* Theoretical Computer Science 1052, 115429 (2025). doi:10.1016/j.tcs.2025.115429. Inspected full text: arXiv:2501.08735v1, Theorems 18–19 and Section 5, Open Problem 2.

[2] Felicia Lucke, Daniël Paulusma, and Bernard Ries. *Finding Matching Cuts in H-Free Graphs.* Algorithmica 85, 3290–3322 (2023). doi:10.1007/s00453-023-01137-9. Theorem 21 gives the radius-two algorithm.

A. Verification and reproducibility

The candidate uses Python 3.14 and the standard library. Checking additionally requires NetworkX and Z3; paper compilation uses Typst. Commands below run from `campaigns/bipartite-radius-three-perfect-matching-cut/` in the repository.

```
python3 work/algorithm.py < source.json > target.json
python3 work/algorithm.py --extract < recovery-input.json
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
python3 work/evidence/parser-totality/check.py
cd work
typst compile manuscript.typ manuscript.pdf
```

A source object contains `n`, `variables` (the explicit list of integer labels) and `clauses` (triples of distinct variable labels). Recovery input contains `source` and `target_solution`; the latter is a full list of integer color bits. The CLI disables Python's fixed decimal-integer digit limit before parsing, so large integer strings reach ordinary arbitrary-precision source validation. Excessive nesting cannot encode a legal source object and maps to the designated NO instance. Extraction rejects malformed premises.

The six prepared cases passed with 28 recovered witnesses. An independently implemented additional checker evaluated 38 source instances, including three NO instances, using source enumeration through eight variables. Its 140 target solver calls produced 134 checked recoveries, with at most four witnesses per instance and maximum target order 2628. These suites passed again after the parser repair. The preserved current evidence is in `work/candidate-evidence/`, `work/evidence/full-rule/` and `work/evidence/parser-totality/`.

The independent reviewer checked all 512 normalized selector states and 1024 pinned binary-interface states. Exactly three and two states, respectively, survived their relaxed boundary contracts. Additional full-graph checks included five source instances, all eight source projections for a one-clause target, three UNSAT bad-projection queries, and 18 fresh recoveries. A focused follow-up checked 15 processes after the parser repair. Reports and original outputs are in `reviews/complete-rule/` and `reviews/parser-followup/`. The latter permits progress to expert review, with the documented novelty-coverage limitations.

These are finite implementation checks. The general interface, composition and recovery arguments establish the mathematical claim for arbitrary legal inputs. No solver, process or campaign timeouts are used; enumerations and test sets have explicit finite scopes. Figures are generated from saved explicit gadget edges by `work/figures/draw.py`, which also checks the displayed cut at every vertex.